```
import React, { useState, useEffect, useRef } from 'react';
import {
  Card,
  Table,
  Button,
  Modal,
  Form,
  Input,
  Select,
  Radio,
  Checkbox,
  DatePicker,
  message,
  Tag,
  Typography,
  Row,
  Col,
  Divider,
  Spin,
  Tooltip,
  Popconfirm,
  notification
} from 'antd';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip as ReTooltip, Legend,
ResponsiveContainer } from 'recharts';
import { PlusOutlined, EditOutlined, DeleteOutlined, ExportOutlined,
ShareAltOutlined, EyeOutlined } from '@ant-design/icons';
const { Title, Text } = Typography;
const { Option } = Select;
const { RangePicker } = DatePicker;
const QuestionnaireManagementModule = () => {
  const [form] = Form.useForm();
  const [analysisForm] = Form.useForm();
  const [loading, setLoading] = useState(false);
  const [modalVisible, setModalVisible] = useState(false);
  const [currentQuestionnaire, setCurrentQuestionnaire] = useState(null);
  const [activeTab, setActiveTab] = useState('list');
  const [questionnaires, setQuestionnaires] = useState([
    {
      id: 1,
      title: '生活习惯问卷',
      type: 'lifestyle',
      createdAt: '2025-09-10',
      status: 'active',
      responses: 124,
      lastResponse: '2025-11-05'
    },
    {
      id: 2,
      title: '饮食结构评估',
```

```
      type: 'diet',
      createdAt: '2025-08-15',
      status: 'active',
      responses: 89,
      lastResponse: '2025-11-03'
    },
    {
      id: 3,
      title: '运动频率调查',
      type: 'exercise',
      createdAt: '2025-07-22',
      status: 'inactive',
      responses: 67,
      lastResponse: '2025-10-28'
    }
  ]);
  const [questions, setQuestions] = useState([]);
  const [responses, setResponses] = useState([]);
  const [analysisData, setAnalysisData] = useState([]);
  const [selectedQuestionnaireId, setSelectedQuestionnaireId] = useState(null);
  const chartRef = useRef();
  useEffect(() => {
    if (selectedQuestionnaireId) {
      loadResponses(selectedQuestionnaireId);
      generateAnalysis(selectedQuestionnaireId);
    }
  }, [selectedQuestionnaireId]);
  const questionnaireTypes = [
    { label: '生活习惯', value: 'lifestyle' },
    { label: '饮食结构', value: 'diet' },
    { label: '运动频率', value: 'exercise' },
    { label: '睡眠质量', value: 'sleep' },
    { label: '心理状态', value: 'mental' }
  ];
  const defaultQuestions = {
    lifestyle: [
      { id: 1, text: '您的作息是否规律？', type: 'radio', options: ['是', '否'] },
      { id: 2, text: '平均每天使用电子设备时长？', type: 'select', options: ['<2 小时', '2-4 小时', '4-6 小时', '>6 小时'] },
      { id: 3, text: '是否有熬夜习惯？', type: 'radio', options: ['经常', '偶尔', '从不'] }
    ],
    diet: [
      { id: 1, text: '每日蔬菜摄入量是否达标？', type: 'radio', options: ['是', '否'] },
      { id: 2, text: '每周红肉食用次数？', type: 'select', options: ['0-1 次', '2-3 次', '4-5 次', '6 次以上'] },
      { id: 3, text: '是否常吃外卖？', type: 'radio', options: ['是', '否'] }
    ],
    exercise: [
```

```
      { id: 1, text: '每周运动次数？', type: 'select', options: ['0 次', '1-2 次', '3-4 次', '5 次及以上'] },
      { id: 2, text: '单次运动时长？', type: 'select', options: ['<30 分钟', '30-60 分钟', '>60 分钟'] },
      { id: 3, text: '主要运动类型？', type: 'checkbox', options: ['跑步', '健身', '瑜伽', '球类', '其他'] }
    ]
  };
  const showModal = (record = null) => {
    setCurrentQuestionnaire(record);
    form.resetFields();
    if (record) {
      form.setFieldsValue(record);
      setQuestions(defaultQuestions[record.type] || []);
    } else {
      setQuestions([]);
    }
    setModalVisible(true);
  };
  const handleCancel = () => {
    setModalVisible(false);
    setCurrentQuestionnaire(null);
  };
  const handleCreate = async (values) => {
    setLoading(true);
    try {
      const newQn = {
        id: questionnaires.length + 1,
        ...values,
        createdAt: new Date().toISOString().split('T')[0],
        status: 'active',
        responses: 0
      };
      setQuestionnaires([...questionnaires, newQn]);
      message.success('问卷创建成功！');
      handleCancel();
    } catch (error) {
      message.error('创建失败，请重试');
    } finally {
      setLoading(false);
    }
  };
  const handleUpdate = async (values) => {
    setLoading(true);
    try {
      const updated = questionnaires.map(qn =>
        qn.id === currentQuestionnaire.id ? { ...qn, ...values } : qn
      );
      setQuestionnaires(updated);
      message.success('问卷更新成功！');
```

```
      handleCancel();
    } catch (error) {
      message.error('更新失败');
    } finally {
      setLoading(false);
    }
  };
  const handleDelete = (id) => {
    setQuestionnaires(questionnaires.filter(qn => qn.id !== id));
    message.success('删除成功');
    if (selectedQuestionnaireId === id) {
      setSelectedQuestionnaireId(null);
      setResponses([]);
      setAnalysisData([]);
    }
  };
  const handleEdit = (record) => {
    showModal(record);
  };
  const columns = [
    {
      title: '问卷标题',
      dataIndex: 'title',
      key: 'title',
      render: (text, record) => (
        <div>
          <Text strong>{text}</Text>
          <br />
          <Tag color={record.status === 'active' ? 'green' : 'default'}>
            {record.status === 'active' ? '启用中' : '已停用'}
          </Tag>
        </div>
      )
    },
    {
      title: '类型',
      dataIndex: 'type',
      key: 'type',
      render: (type) => {
        const typeMap = {
          lifestyle: '生活习惯',
          diet: '饮食结构',
          exercise: '运动频率',
          sleep: '睡眠质量',
          mental: '心理状态'
        };
        return typeMap[type] || type;
      }
    },
    {
```

```
      title: '创建时间',
      dataIndex: 'createdAt',
      key: 'createdAt'
    },
    {
      title: '响应数量',
      dataIndex: 'responses',
      key: 'responses',
      sorter: (a, b) => a.responses - b.responses,
      render: (count) => <Tag color="blue">{count}人</Tag>
    },
    {
      title: '最后响应',
      dataIndex: 'lastResponse',
      key: 'lastResponse'
    },
    {
      title: '操作',
      key: 'action',
      render: (_, record) => (
        <span>
          <Button    type="link"    icon={<EyeOutlined    />}    onClick={()    =>
viewResponses(record.id)}>
            查看
          </Button>
          <Button    type="link"    icon={<EditOutlined    />}    onClick={()    =>
handleEdit(record)}>
            编辑
          </Button>
          <Popconfirm
            title="确定要删除此问卷吗？"
            onConfirm={() => handleDelete(record.id)}
            okText="确定"
            cancelText="取消"
          >
            <Button type="link" danger icon={<DeleteOutlined />}>
              删除
            </Button>
          </Popconfirm>
        </span>
      )
    }
  ];
  const loadResponses = (qid) => {
    const mockResponses = [
      {
        id: 1,
        userId: 'U001',
        submittedAt: '2025-11-05 14:30',
        answers: {
```

```
        1: '是',
        2: '2-4 小时',
        3: '偶尔'
      }
    },
    {
      id: 2,
      userId: 'U002',
      submittedAt: '2025-11-04 09:15',
      answers: {
        1: '否',
        2: '>6 小时',
        3: '经常'
      }
    }
  ];
  setResponses(mockResponses);
};
const generateAnalysis = (qid) => {
  const data = [
    { name: '周一', 提交数: 12 },
    { name: '周二', 提交数: 18 },
    { name: '周三', 提交数: 15 },
    { name: '周四', 提交数: 22 },
    { name: '周五', 提交数: 28 },
    { name: '周六', 提交数: 20 },
    { name: '周日', 提交数: 10 }
  ];
  setAnalysisData(data);
};
const viewResponses = (qid) => {
  setSelectedQuestionnaireId(qid);
  setActiveTab('analysis');
};
const exportData = () => {
  notification.success({
    message: '导出成功',
    description: '问卷数据已导出为 Excel 文件。'
  });
};
const shareData = () => {
  notification.info({
    message: '分享提示',
    description: '复制链接或通过邮件分享给医生或家人。'
  });
};
// const previewQuestionnaire = (qid) => {
//   window.open(`/preview/${qid}`, '_blank');
// };
const addDefaultQuestions = (type) => {
```

```
    if (defaultQuestions[type]) {
      setQuestions([...defaultQuestions[type]]);
    }
  };
  const addCustomQuestion = () => {
    const newQ = {
      id: questions.length + 1,
      text: '',
      type: 'text',
      options: []
    };
    setQuestions([...questions, newQ]);
  };
  const removeQuestion = (qid) => {
    setQuestions(questions.filter(q => q.id !== qid));
  };
  const updateQuestion = (qid, field, value) => {
    setQuestions(questions.map(q => q.id === qid ? { ...q, [field]: value } : q));
  };
  const submitQuestionnaireSetup = () => {
    const values = form.getFieldsValue();
    if (currentQuestionnaire) {
      handleUpdate(values);
    } else {
      handleCreate(values);
    }
  };
  const responseColumns = [
    {
      title: '用户 ID',
      dataIndex: 'userId',
      key: 'userId'
    },
    {
      title: '提交时间',
      dataIndex: 'submittedAt',
      key: 'submittedAt'
    },
    {
      title: '回答详情',
      key: 'answers',
      render: (text, record) => (
        <div>
          {Object.entries(record.answers).map(([qid, ans]) => {
            const q = questions.find(q => q.id == qid);
            return q ? <p key={qid}><strong>{q.text}:</strong> {ans}</p> : null;
          })}
        </div>
      )
    }
```

```
  ];
  if (activeTab === 'list') {
    return (
      <Card
        title="问卷调查管理"
        extra={
          <Button   type="primary"   icon={<PlusOutlined   />}   onClick={()   =>
showModal()}>
            新建问卷
          </Button>
        }
      >
        <Table
          dataSource={questionnaires}
          columns={columns}
          rowKey="id"
          pagination={{ pageSize: 10 }}
        />
        <Modal
          title={currentQuestionnaire ? '编辑问卷' : '创建新问卷'}
          open={modalVisible}
          onOk={submitQuestionnaireSetup}
          onCancel={handleCancel}
          confirmLoading={loading}
          width={800}
        >
          <Form form={form} layout="vertical">
            <Row gutter={16}>
              <Col span={12}>
                <Form.Item
                  name="title"
                  label="问卷标题"
                  rules={[{ required: true, message: '请输入问卷标题' }]}
                >
                  <Input placeholder="例如：生活习惯评估" />
                </Form.Item>
              </Col>
              <Col span={12}>
                <Form.Item
                  name="type"
                  label="问卷类型"
                  rules={[{ required: true, message: '请选择类型' }]}
                >
                  <Select onChange={addDefaultQuestions} placeholder="选择问卷类型
">
                    {questionnaireTypes.map(type => (
                      <Option                                    key={type.value}
value={type.value}>{type.label}</Option>
                    ))}
                  </Select>
```

```
                </Form.Item>
              </Col>
            </Row>
            <Divider>问题设置</Divider>
            <div style={{ maxHeight: '300px', overflowY: 'auto', marginBottom: 16 }}>
              {questions.map((q) => (
                <Card key={q.id} size="small" style={{ marginBottom: 8 }}>
                  <Row align="middle">
                    <Col flex="auto">
                      <Input
                        value={q.text}
                        onChange={(e)     =>     updateQuestion(q.id,     'text',
e.target.value)}
                        placeholder="问题内容"
                        style={{ marginBottom: 8 }}
                      />
                      <Select
                        value={q.type}
                        onChange={(val) => updateQuestion(q.id, 'type', val)}
                        style={{ width: 120, marginRight: 8 }}
                      >
                        <Option value="text">文本</Option>
                        <Option value="radio">单选</Option>
                        <Option value="checkbox">多选</Option>
                        <Option value="select">下拉选择</Option>
                      </Select>
                      {(q.type === 'radio' || q.type === 'checkbox' || q.type ===
'select') && (
                        <Input.TextArea
                          value={q.options.join(',')}
                          onChange={(e)    =>    updateQuestion(q.id,    'options',
e.target.value.split(','))}
                          placeholder="选项，用逗号分隔"
                          rows={1}
                        />
                      )}
                    </Col>
                    <Col>
                      <Popconfirm
                        title="确定删除此问题？"
                        onConfirm={() => removeQuestion(q.id)}
                      >
                        <Button type="text" danger>删除</Button>
                      </Popconfirm>
                    </Col>
                  </Row>
                </Card>
              ))}
            </div>
            <Button type="dashed" onClick={addCustomQuestion} block>
```

```
              + 添加自定义问题
            </Button>
          </Form>
        </Modal>
      </Card>
    );
  }
  if (activeTab === 'analysis') {
    const selectedQn = questionnaires.find(q => q.id === selectedQuestionnaireId);
    return (
      <div>
        <Card style={{ marginBottom: 16 }}>
          <Title level={3}>{selectedQn?.title}</Title>
          <Text type="secondary">类型：{selectedQn?.type}</Text>
          <div style={{ marginTop: 16 }}>
            <Button      icon={<ExportOutlined      />}      onClick={exportData}
style={{ marginRight: 8 }}>
              导出数据
            </Button>
            <Button icon={<ShareAltOutlined />} onClick={shareData}>
              分享
            </Button>
            <Button style={{ marginLeft: 8 }} onClick={() => setActiveTab('list')}>
              返回列表
            </Button>
          </div>
        </Card>
        <Row gutter={16}>
          <Col span={16}>
            <Card title="提交趋势分析">
              <ResponsiveContainer width="100%" height={300}>
                <LineChart data={analysisData}>
                  <CartesianGrid strokeDasharray="3 3" />
                  <XAxis dataKey="name" />
                  <YAxis />
                  <ReTooltip />
                  <Legend />
                  <Line type="monotone" dataKey="提交数" stroke="#1890ff" />
                </LineChart>
              </ResponsiveContainer>
            </Card>
          </Col>
          <Col span={8}>
            <Card title="统计摘要">
              <p><Text strong>总响应数：</Text> {responses.length}</p>
              <p><Text strong>最近提交：</Text> {responses.length > 0 ?
responses[0].submittedAt : '无'}</p>
              <p><Text strong>平均完成时长：</Text> 约 3.2 分钟</p>
            </Card>
          </Col>
```

```
        </Row>
        <Card title="详细响应记录" style={{ marginTop: 16 }}>
          <Table
            dataSource={responses}
            columns={responseColumns}
            rowKey="id"
            pagination={{ pageSize: 10 }}
            scroll={{ y: 300 }}
          />
        </Card>
      </div>
    );
  }
  return null;
};
export default QuestionnaireManagementModule;
import React, { useState, useEffect } from 'react';
import {
  Layout,
  Menu,
  Card,
  Typography,
  Button,
  List,
  Avatar,
  Modal,
  Form,
  Input,
  Select,
  Rate,
  Divider,
  Tag,
  Space,
  message,
  Spin,
  Row,
  Col,
  Image,
  Carousel,
} from 'antd';
import {
  HomeOutlined,
  ReadOutlined,
  VideoCameraOutlined,
  QuestionCircleOutlined,
  UserOutlined,
  SearchOutlined,
  PlusOutlined,
  StarOutlined,
  CheckCircleOutlined,
```

```
  CloseCircleOutlined,
} from '@ant-design/icons';
const { Header, Content, Sider } = Layout;
const { Title, Text, Paragraph } = Typography;
const { Option } = Select;
const { TextArea } = Input;
// 模拟用户信息
const currentUser = {
  id: 1,
  name: '张三',
  avatar: 'https://randomuser.me/api/portraits/men/32.jpg',
  level: '轻度亚健康',
  risk: '中等',
};
// 模拟资讯数据
const mockArticles = [
  {
    id: 1,
    title: '如何科学缓解工作压力？',
    summary: '长期高压工作容易导致亚健康，本文介绍几种科学减压方法。',
    author: '李医生',
    date: '2025-10-20',
    category: '心理健康',
    readCount: 1245,
    cover: 'https://via.placeholder.com/300x180?text=压力管理',
    type: 'article',
  },
  {
    id: 2,
    title: '办公室久坐族的5个自救动作',
    summary: '每天只需10分钟，改善血液循环，预防颈椎和腰椎问题。',
    author: '王康复师',
    date: '2025-10-18',
    category: '运动健康',
    readCount: 2103,
    cover: 'https://via.placeholder.com/300x180?text=办公室运动',
    type: 'video',
  },
  {
    id: 3,
    title: '亚健康人群饮食调整指南',
    summary: '合理膳食结构是改善亚健康的基石，建议增加膳食纤维摄入。',
    author: '赵营养师',
    date: '2025-10-15',
    category: '营养健康',
    readCount: 1876,
    cover: 'https://via.placeholder.com/300x180?text=健康饮食',
    type: 'article',
  },
];
```

```
// 模拟问答数据
const mockQuestions = [
  {
    id: 1,
    question: '经常失眠怎么办？',
    answer:
      '建议保持规律作息，避免睡前使用电子设备。可尝试冥想或温水泡脚。如持续超过两周，建议就医评估。',
    answeredBy: '刘心理医生',
    date: '2025-10-19',
    rating: 4.5,
  },
  {
    id: 2,
    question: '体检正常但总感觉累，是亚健康吗？',
    answer:
      '是的，这可能是典型的亚健康状态。建议进行生活方式评估，并适当调整饮食与运动习惯。',
    answeredBy: '陈内科医生',
    date: '2025-10-17',
    rating: 5,
  },
];
// 内容卡片组件
const ContentCard = ({ item, onClick }) => (
  <List.Item onClick={() => onClick(item)}>
    <Card hoverable style={{ width: '100%' }}>
      <Row gutter={16}>
        <Col span={6}>
          <Image src={item.cover} alt={item.title} style={{ width: '100%', height: 120, objectFit: 'cover', borderRadius: 8 }} />
        </Col>
        <Col span={18}>
          <Title level={5} style={{ margin: 0 }}>{item.title}</Title>
          <Text type="secondary" ellipsis={{ rows: 1 }}>{item.summary}</Text>
          <div style={{ marginTop: 8, display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
            <Space size="small">
              <Tag color="blue">{item.category}</Tag>
              <Text type="secondary" style={{ fontSize: 12 }}>
                {item.author} • {item.date}
              </Text>
            </Space>
            <Text type="secondary" style={{ fontSize: 12 }}>
              阅读 {item.readCount}
            </Text>
          </div>
        </Col>
      </Row>
    </Card>
```

```
  </List.Item>
);
// 问答项组件
const QuestionItem = ({ item }) => (
  <List.Item key={item.id}>
    <Card style={{ width: '100%' }}>
      <div>
        <Text strong>{item.question}</Text>
        <Paragraph style={{ marginTop: 8, fontSize: 14 }}>{item.answer}</Paragraph>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginTop: 8 }}>
          <Text type="secondary" style={{ fontSize: 12 }}>
            回答：{item.answeredBy} • {item.date}
          </Text>
          <Rate disabled allowHalf value={item.rating} />
        </div>
      </div>
    </Card>
  </List.Item>
);
// 健康教育系统主页面
const HealthEducationSystem = () => {
  const [selectedMenu, setSelectedMenu] = useState('home');
  const [visible, setVisible] = useState(false);
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(true);
  const [articles, setArticles] = useState([]);
  const [questions, setQuestions] = useState([]);
  useEffect(() => {
    // 模拟加载数据
    setTimeout(() => {
      setArticles(mockArticles);
      setQuestions(mockQuestions);
      setLoading(false);
    }, 800);
  }, []);
  const handleMenuClick = (e) => {
    setSelectedMenu(e.key);
  };
  const showModal = () => {
    setVisible(true);
  };
  const handleCancel = () => {
    setVisible(false);
    form.resetFields();
  };
  const handleFinish = (values) => {
    const newQuestion = {
      id: questions.length + 1,
      question: values.question,
```

```
      answer: '感谢提问，您的问题已提交，专业医生将在 24 小时内为您解答。',
      answeredBy: '系统自动回复',
      date: new Date().toISOString().split('T')[0],
      rating: 0,
    };
    setQuestions([newQuestion, ...questions]);
    message.success('提问成功，我们将尽快为您解答！');
    setVisible(false);
    form.resetFields();
  };
  const handleContentClick = (item) => {
    Modal.info({
      title: item.title,
      content: (
        <div>
          <p><strong>作者：</strong>{item.author}</p>
          <p><strong>类别：</strong>{item.category}</p>
          <p><strong>发布日期：</strong>{item.date}</p>
          <Divider />
          <p>{item.summary}</p>
          {/*
          if (item.type === 'video') {
            return <video controls style={{ width: '100%' }}><source
src={item.videoUrl} type="video/mp4" />></video>;
          }
          */}
        </div>
      ),
      onOk() {},
      width: 640,
    });
  };
  const renderHome = () => (
    <div>
      <Carousel autoplay>
        <div>
          <Card style={{ textAlign: 'center', background: '#e6f7ff', borderRadius:
8 }}>
            <Title level={3}>关注亚健康，从今天开始</Title>
    fetchReport();
  }, []);
  // const handleManualRefresh = () => {
  //   setLoading(true);
  //   setTimeout(() => {
  //     message.success('报告已更新');
  //     setLoading(false);
  //   }, 1000);
  // };
  const handleGenerateNewReport = () => {
    setLoading(true);
```

```
    // 模拟重新生成报告
    setTimeout(() => {
      message.success('新的生活方式分析报告已生成');
      setLoading(false);
    }, 1200);
  };
  const showFeedbackModal = () => {
    setVisible(true);
  };
  const handleFeedbackSubmit = async () => {
    try {
      const values = await feedbackForm.validateFields();
      console.log('提交的反馈:', values);
      message.success('感谢您的反馈，我们将持续优化报告内容。');
      setVisible(false);
      feedbackForm.resetFields();
      // 更新报告中的反馈统计
      setReport((prev) => ({
        ...prev,
        feedbackCount: prev.feedbackCount + 1,
        lastFeedback: new Date().toISOString().split('T')[0],
      }));
    } catch (errorInfo) {
      console.log('验证失败:', errorInfo);
    }
  };
  const handleCancel = () => {
    setVisible(false);
  };
  const getScoreColor = (score) => {
    if (score >= 80) return 'green';
    if (score >= 60) return 'orange';
    return 'red';
  };
  const getLevelColor = (level) => {
    switch (level) {
      case 'success':
        return 'green';
      case 'warning':
        return 'orange';
      case 'danger':
        return 'red';
      default:
        return 'blue';
    }
  };
  const columns = [
    {
      title: '风险因素',
      dataIndex: 'factor',
```

```
      key: 'factor',
    },
    {
      title: '风险等级',
      dataIndex: 'level',
      key: 'level',
      render: (text) => <Tag color={text === '高' ? 'red' : text === '中等' ? 'orange' :
'green'}>{text}</Tag>,
    },
    {
      title: '趋势',
      dataIndex: 'trend',
      key: 'trend',
      render: (text) => (
        <span style={{ color: text === '上升' ? '#cf1322' : text === '下降' ?
'#389e0d' : '#1890ff' }}>
          {text === '上升' ? '↑' : text === '下降' ? '↓' : '→'} {text}
        </span>
      ),
    },
  ];
  const recommendationIcon = (level) => {
    switch (level) {
      case 'success':
        return <CheckCircleOutlined style={{ color: '#52c41a' }} />;
      case 'warning':
        return <ExclamationCircleOutlined style={{ color: '#faad14' }} />;
      default:
        return <ExclamationCircleOutlined style={{ color: '#ff4d4f' }} />;
    }
  };
  const scoreData = useMemo(() => {
    return report
      ? [
          { type: '睡眠', value: report.scores.sleep },
          { type: '饮食', value: report.scores.diet },
          { type: '运动', value: report.scores.exercise },
          { type: '饮水', value: report.scores.hydration },
          { type: '屏幕时间', value: report.scores.screenTime },
        ]
      : [];
  }, [report]);
  // const handleDateChange = (dates) => {
  //   if (dates) {
  //     setDateRange(dates);
  //           message.info(` 正 在 加 载  ${dates[0].format('YYYY-MM')}  至
${dates[1].format('YYYY-MM')} 的数据`);
  //   } else {
  //     setDateRange([]);
  //   }
```

```
  // };
  const tabList = [
    {
      key: 'summary',
      tab: '综合概览',
    },
    {
      key: 'details',
      tab: '详细分析',
    },
    {
      key: 'risks',
      tab: '风险预测',
    },
  ];
  const onTabChange = (key) => {
    setActiveTab(key);
  };
  const renderSummaryContent = () => (
    <>
      <Row gutter={[16, 16]} style={{ marginBottom: 24 }}>
        <Col span={24}>
          <Card title="总体健康评分" extra={<FileTextOutlined />}>
            <div style={{ textAlign: 'center', padding: '20px 0' }}>
              <Statistic
                title="综合评分"
                value={report?.scores.overall}
                precision={0}
                valueStyle={{ color: getScoreColor(report?.scores.overall) }}
              />
              <Progress
                percent={report?.scores.overall}
                status="active"
                strokeColor={getScoreColor(report?.scores.overall)}
                style={{ marginTop: 16 }}
              />
              <Text type="secondary" style={{ display: 'block', marginTop: 8 }}>
                报告周期：{report?.period.start} 至 {report?.period.end}
              </Text>
            </div>
          </Card>
        </Col>
      </Row>
      <Row gutter={[16, 16]}>
        {scoreData.map((item) => (
          <Col xs={12} sm={8} md={8} lg={8} xl={8} key={item.type}>
            <Card size="small" title={item.type} bordered={false}>
              <Progress            type="circle"            percent={item.value}
strokeColor={getScoreColor(item.value)} width={80} />
              <div style={{ marginTop: 8 }}>
```

```
                <Text strong>{item.value}</Text>
                <Text type="secondary"> 分</Text>
              </div>
            </Card>
          </Col>
        ))}
      </Row>
      <Divider orientation="left">改善建议</Divider>
      <Card>
        {report?.recommendations.map((rec, index) => (
          <div key={index} style={{ marginBottom: 16, paddingBottom: 16, borderBottom:
'1px dashed #e8e8e8' }}>
            <div style={{ display: 'flex', alignItems: 'flex-start' }}>
              <div        style={{        marginRight:        8,        fontSize:
18 }}>{recommendationIcon(rec.level)}</div>
              <div style={{ flex: 1 }}>
                <Title level={5} style={{ margin: 0, color: getLevelColor(rec.level)
=== 'green' ? '#52c41a' : getLevelColor(rec.level) === 'orange' ? '#faad14' :
'#ff4d4f' }}>
                  {rec.title}
                </Title>
                <Paragraph type="secondary" style={{ margin: '4px 0 0 0' }}>
                  {rec.content}
                </Paragraph>
              </div>
            </div>
          </div>
        ))}
      </Card>
    </>
  );
  const renderDetailsContent = () => (
    <div>
      <Row gutter={[16, 16]} style={{ marginBottom: 24 }}>
        <Col span={24}>
          <Card title="生活习惯详情" extra={<PieChartOutlined />}>
            <Paragraph>
              本报告基于您在过去一个月内的智能设备、手动记录及问卷调查数据进行综合
分析。
              数据涵盖睡眠质量、饮食记录、运动轨迹、饮水提醒等多个维度。
            </Paragraph>
            <ul>
              <li>睡眠数据来源：智能手环/手表、手动打卡</li>
              <li>饮食记录：APP 内拍照识别、手动输入</li>
              <li>运动数据：GPS 轨迹、步数统计</li>
              <li>饮水提醒：每日定时提醒与记录</li>
            </ul>
          </Card>
        </Col>
      </Row>
```

```
      <Card title="用户反馈情况" extra={<CommentOutlined />}>
        <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between' }}>
          <div>
            <Statistic title="累计反馈次数" value={report?.feedbackCount} />
            <Text type="secondary">上次反馈时间：{report?.lastFeedback}</Text>
          </div>
          <Button type="primary" onClick={showFeedbackModal}>
            提交新反馈
          </Button>
        </div>
      </Card>
    </div>
  );
  const renderRisksContent = () => (
    <div>
      <Card title="健康风险预测" extra={<ExclamationCircleOutlined />} style={{ marginBottom: 24 }}>
        <Paragraph>
          基于您的生活习惯数据，系统通过机器学习模型预测未来 3-6 个月可能面临的健康风险。
          请关注高风险项并积极调整生活方式。
        </Paragraph>
        <Table
          dataSource={report?.riskPrediction}
          columns={columns}
          pagination={false}
          rowKey="factor"
        />
      </Card>
      <Card title="风险说明">
        <ul>
          <li><strong>高血压风险：</strong>与高盐饮食、缺乏运动、睡眠不足密切相关。</li>
          <li><strong>糖尿病风险：</strong>受饮食结构、体重变化、运动频率影响较大。</li>
          <li><strong>颈椎病风险：</strong>长时间伏案工作、手机使用过多是主要诱因。</li>
        </ul>
      </Card>
    </div>
  );
  const contentList = {
    summary: renderSummaryContent(),
    details: renderDetailsContent(),
    risks: renderRisksContent(),
  };
  return (
    <div style={{ padding: '24px', background: '#f5f5f5', minHeight: '100vh' }}>
      <Card
```

```
        title={
          <div style={{ display: 'flex', alignItems: 'center' }}>
            <StarOutlined style={{ marginRight: 8, color: '#1890ff' }} />
            <span>亚健康用户 - 生活方式分析报告</span>
          </div>
        }
        extra={
          <Space>
            {/* <RangePicker onChange={handleDateChange} placeholder={['开始日期',
'结束日期']} /> */}
            <Tooltip title="刷新报告">
              <Button        icon={<SyncOutlined        spin={loading}        />}
onClick={handleGenerateNewReport} loading={loading} />
            </Tooltip>
          </Space>
        }
        loading={loading}
        style={{ maxWidth: 1000, margin: '0 auto' }}
      >
        {report && (
          <>
            <Card.TabPane        tabList={tabList}        activeTabKey={activeTab}
onTabChange={onTabChange}>
              {contentList[activeTab]}
            </Card.TabPane>
            <Divider />
            <div style={{ textAlign: 'center', marginTop: 24 }}>
              <Text type="secondary">
                报告生成时间：{new Date(report.generatedAt).toLocaleString('zh-CN')}
              </Text>
              <br />
              <Button           type="link"           onClick={showFeedbackModal}
icon={<CommentOutlined />}>
                对本报告提出建议
              </Button>
            </div>
          </>
        )}
      </Card>
      <Modal
        title="提交反馈"
        open={visible}
        onOk={handleFeedbackSubmit}
        onCancel={handleCancel}
        okText="提交"
        cancelText="取消"
      >
        <Form form={feedbackForm} layout="vertical">
          <Form.Item
            name="category"
```

```
            label="反馈类别"
            rules={[{ required: true, message: '请选择反馈类别' }]}
          >
            <Select placeholder="请选择">
              <Option value="accuracy">数据准确性</Option>
              <Option value="suggestion">建议实用性</Option>
              <Option value="interface">界面体验</Option>
              <Option value="other">其他</Option>
            </Select>
          </Form.Item>
          <Form.Item
            name="content"
            label="反馈内容"
            rules={[{ required: true, message: '请输入反馈内容' }]}
          >
            <Input.TextArea rows={4} placeholder="请描述您的意见或建议..." />
          </Form.Item>
        </Form>
      </Modal>
    </div>
  );
};
// const AppWrapper = () => (
//   <ConfigProvider locale={zhCN}>
//     <LifestyleAnalysisModule />
//   </ConfigProvider>
// );
export default LifestyleAnalysisModule;
import React, { useState, useEffect, useCallback } from 'react';
import {
  Card,
  Typography,
  Progress,
  Button,
  Rate,
  List,
  Avatar,
  Modal,
  Form,
  Input,
  message,
  Tag,
  Row,
  Col,
  Divider,
  Spin,
  Select,
  DatePicker,
} from 'antd';
import moment from 'moment';
```

```
import _ from 'lodash';
const { Title, Text, Paragraph } = Typography;
const { Option } = Select;
const { TextArea } = Input;
// 健康报告生成主页面组件
const HealthReportGenerator = () => {
  const [reports, setReports] = useState([]);
  const [loading, setLoading] = useState(true);
  const [visible, setVisible] = useState(false);
  const [currentReport, setCurrentReport] = useState(null);
  const [feedbackForm] = Form.useForm();
  const [activeTab, setActiveTab] = useState('latest');
  const [filterDate, setFilterDate] = useState(null);
  // 模拟从后端获取用户健康报告数据
  useEffect(() => {
    const fetchReports = async () => {
      setLoading(true);
      // 模拟 API 调用延迟
      await new Promise((resolve) => setTimeout(resolve, 800));
      const mockData = [
        {
          id: 1,
          date: '2025-11-01',
          score: 76,
          category: 'monthly',
          status: 'completed',
          summary: '整体健康状况良好，但需注意睡眠质量和饮食均衡。',
          habits: {
            sleep: { value: 6.5, target: 7.5, unit: '小时' },
            diet: { value: 3, target: 5, unit: '蔬果/天' },
            exercise: { value: 2, target: 3, unit: '次/周' },
            water: { value: 1.8, target: 2, unit: '升/天' },
          },
          risks: ['轻度疲劳', '维生素缺乏风险'],
          suggestions: [
            '建议每晚提前半小时入睡，逐步调整至 7.5 小时',
            '增加深色蔬菜摄入，如菠菜、西兰花',
            '每周至少进行 3 次中等强度运动，如快走或骑行',
          ],
          feedbackCount: 2,
          lastFeedback: '报告建议很实用，已开始改善作息',
        },
        {
          id: 2,
          date: '2025-10-01',
          score: 68,
          category: 'monthly',
          status: 'completed',
          summary: '亚健康状态较明显，主要问题集中在体力下降和情绪波动。',
          habits: {
```

```
          sleep: { value: 5.8, target: 7.5, unit: '小时' },
          diet: { value: 2, target: 5, unit: '蔬果/天' },
          exercise: { value: 1, target: 3, unit: '次/周' },
          water: { value: 1.2, target: 2, unit: '升/天' },
        },
        risks: ['慢性疲劳综合征风险', '免疫力下降'],
        suggestions: [
          '避免熬夜，设定固定的睡觉和起床时间',
          '午餐增加优质蛋白摄入，如鱼、豆制品',
          '尝试每日 10 分钟冥想缓解压力',
        ],
        feedbackCount: 4,
        lastFeedback: '正在尝试冥想，感觉有帮助',
      },
      {
        id: 3,
        date: '2025-09-01',
        score: 82,
        category: 'monthly',
        status: 'completed',
        summary: '健康状态稳步提升，继续保持良好习惯。',
        habits: {
          sleep: { value: 7.2, target: 7.5, unit: '小时' },
          diet: { value: 4, target: 5, unit: '蔬果/天' },
          exercise: { value: 3, target: 3, unit: '次/周' },
          water: { value: 1.9, target: 2, unit: '升/天' },
        },
        risks: [],
        suggestions: [
          '继续保持当前生活习惯',
          '可尝试引入力量训练增强肌肉耐力',
          '定期监测血压与心率变化',
        ],
        feedbackCount: 1,
        lastFeedback: '会继续坚持锻炼',
      },
    ];
    setReports(mockData);
    setLoading(false);
  };
  fetchReports();
}, []);
// 根据筛选条件过滤报告
const filteredReports = useCallback(() => {
  let result = [...reports];
  if (filterDate) {
    const filterMoment = moment(filterDate);
    result = result.filter((r) => moment(r.date).isSame(filterMoment, 'month'));
  }
  if (activeTab === 'improving') {
```

```
    result = result.filter((r) => r.score > 70);
  } else if (activeTab === 'needsAttention') {
    result = result.filter((r) => r.score <= 70);
  }
  return result.sort((a, b) => moment(b.date) - moment(a.date));
}, [reports, activeTab, filterDate]);
// 打开反馈弹窗
const openFeedbackModal = (report) => {
  setCurrentReport(report);
  setVisible(true);
  feedbackForm.resetFields();
};
// 提交反馈
const handleFeedbackSubmit = async (values) => {
  // 模拟提交反馈
  await new Promise((resolve) => setTimeout(resolve, 500));
  setReports((prev) =>
    prev.map((r) =>
      r.id === currentReport.id
        ? {
            ...r,
            feedbackCount: r.feedbackCount + 1,
            lastFeedback: values.content,
          }
        : r
    )
  );
  message.success('感谢您的反馈！我们将持续优化报告质量。');
  setVisible(false);
  feedbackForm.resetFields();
};
// 重新生成报告（模拟操作）
const regenerateReport = async (reportId) => {
  message.loading('正在重新分析数据...', 1.5);
  await new Promise((resolve) => setTimeout(resolve, 1500));
  message.success('报告已更新！');
};
// 导出报告为PDF（模拟）
const exportAsPDF = (reportId) => {
  message.info('正在生成PDF文件...');
  setTimeout(() => {
    message.success('PDF导出成功！');
  }, 1000);
};
// 渲染习惯项
const renderHabitItem = (label, data) => (
  <div style={{ marginBottom: 12 }}>
    <Text strong>{label}：</Text>
    <Text>{data.value.toFixed(1)} {data.unit}</Text>
    <Progress
```

```
        percent={Math.round((data.value / data.target) * 100)}
        size="small"
        status="active"
        strokeColor="#52c41a"
        showInfo={false}
        style={{ marginTop: 4 }}
      />
    </div>
  );
  // const mockSuggestions = [
  //   '建议每天步行 8000 步以上',
  //   '减少精制糖摄入，选择全谷物食品',
  //   '保持室内空气流通，每日通风两次',
  // ];
  return (
    <div style={{ padding: '24px', backgroundColor: '#f5f5f5', minHeight: '100vh' }}>
      <Card bordered={false}>
        <Title level={2} style={{ textAlign: 'center', marginBottom: 32 }}>
          个性化健康报告
        </Title>
        {/* 筛选区域 */}
        <Row gutter={[16, 16]} align="middle" style={{ marginBottom: 24 }}>
          <Col xs={24} sm={12} md={8} lg={6}>
            <DatePicker
              picker="month"
              placeholder="选择月份"
              onChange={setFilterDate}
              allowClear
              style={{ width: '100%' }}
            />
          </Col>
          <Col xs={24} sm={12} md={8} lg={6}>
            <Select
              value={activeTab}
              onChange={setActiveTab}
              style={{ width: '100%' }}
            >
              <Option value="latest">全部报告</Option>
              <Option value="improving">健康改善中</Option>
              <Option value="needsAttention">需关注状态</Option>
            </Select>
          </Col>
        </Row>
        {/* 报告列表 */}
        <Spin spinning={loading}>
          {filteredReports().length === 0 ? (
            <div style={{ textAlign: 'center', color: '#999', padding: 40 }}>
              <Text>暂无符合条件的健康报告</Text>
            </div>
          ) : (
```

```
            <List
              dataSource={filteredReports()}
              renderItem={(report) => (
                <List.Item key={report.id}>
                  <List.Item.Meta
                    avatar={
                      <Avatar
                        style={{
                          backgroundColor: report.score >= 70 ? '#52c41a' : '#ff4d4f',
                        }}
                      >
                        {report.score}
                      </Avatar>
                    }
                    title={
                      <div>
                        <Text strong>健康评分：{report.score}/100</Text>
                        <Tag style={{ marginLeft: 8 }}>
                          {moment(report.date).format('YYYY年MM月')}
                        </Tag>
                        {report.score > 70 && (
                          <Tag color="success" style={{ marginLeft: 8 }}>
                            健康改善
                          </Tag>
                        )}
                      </div>
                    }
                    description={
                      <div>
                        <Paragraph          style={{          margin:          '8px
0' }}>{report.summary}</Paragraph>
                        <div style={{ display: 'flex', flexWrap: 'wrap', gap: 16,
marginTop: 12 }}>
                          {Object.entries(report.habits).map(([key, data]) => (
                            <div key={key} style={{ minWidth: 120 }}>
                              <Text type="secondary">
                                {{
                                  sleep: '睡眠',
                                  diet: '饮食',
                                  exercise: '运动',
                                  water: '饮水',
                                }[key]}
                              </Text>
                              <br />
                              <Text strong>{data.value.toFixed(1)}</Text>
                              <Text type="secondary">/{data.target}</Text>
                            </div>
                          ))}
                        </div>
                        {report.risks.length > 0 && (
```

```
                        <div style={{ marginTop: 12 }}>
                          <Text type="danger">⚠ 风险提示：</Text>
                          {report.risks.map((risk, i) => (
                            <Tag key={i} color="warning" style={{ marginLeft:
8 }}>
                              {risk}
                            </Tag>
                          ))}
                        </div>
                      )}
                      <Divider dashed style={{ margin: '12px 0' }} />
                      <div>
                        <Button
                          type="link"
                          size="small"
                          onClick={() => openFeedbackModal(report)}
                        >
                          提交反馈
                        </Button>
                        <Button
                          type="link"
                          size="small"
                          onClick={() => regenerateReport(report.id)}
                        >
                          重新生成
                        </Button>
                        <Button
                          type="link"
                          size="small"
                          onClick={() => exportAsPDF(report.id)}
                        >
                          导出 PDF
                        </Button>
                      </div>
                      {report.lastFeedback && (
                        <div style={{ marginTop: 8, padding: '8px 0', borderTop:
'1px dashed #e8e8e8' }}>
                          <Text italic>上次反馈：{report.lastFeedback}</Text>
                        </div>
                      )}
                    </div>
                  }
                />
                {/* 展开详情 */}
                <div style={{ width: '100%', marginTop: 16 }}>
                  <Title level={4}>详细建议</Title>
                  <List
                    size="small"
                    dataSource={report.suggestions}
                    renderItem={(item, index) => (
```

```
                    <List.Item>
                      <Text>{index + 1}. {item}</Text>
                    </List.Item>
                  )}
                />
                {/* 健康习惯进度 */}
                <Title level={4} style={{ marginTop: 24 }}>习惯完成情况</Title>
                {renderHabitItem('睡眠时长', report.habits.sleep)}
                {renderHabitItem('蔬果摄入', report.habits.diet)}
                {renderHabitItem('运动频率', report.habits.exercise)}
                {renderHabitItem('饮水量', report.habits.water)}
              </div>
            </List.Item>
          )}
        />
      )}
    </Spin>
    {/* 反馈模态框 */}
    <Modal
      title="提交反馈"
      open={visible}
      onCancel={() => setVisible(false)}
      footer={null}
      destroyOnClose
    >
      <Form form={feedbackForm} onFinish={handleFeedbackSubmit}>
        <Form.Item
          name="rating"
          label="您对本次报告的满意度"
          rules={[{ required: true, message: '请选择评分' }]}
        >
          <Rate />
        </Form.Item>
        <Form.Item
          name="content"
          label="您的建议或意见"
          rules={[{ required: true, message: '请输入反馈内容' }]}
        >
          <TextArea rows={4} placeholder="例如：报告建议很实用，但我希望看到更多心理健康的分析..." />
        </Form.Item>
        <Form.Item>
          <Button type="primary" htmlType="submit" block>
            提交反馈
          </Button>
        </Form.Item>
      </Form>
    </Modal>
  </Card>
  {/* 页面底部说明 */}
```

```
      <div style={{ textAlign: 'center', marginTop: 32, color: '#999' }}>
        <Text>
          系统基于您的生活习惯数据自动生成报告，建议每月查看一次以跟踪健康趋势。
        </Text>
        <br />
        <Text type="secondary">
          数据更新时间：{moment().format('YYYY-MM-DD HH:mm')}
        </Text>
      </div>
      {/*
      // 将来可能加入的功能：
      // - 多维度雷达图展示健康指标
      // - 与医生在线沟通入口
      // - 家庭成员报告对比
      */}
    </div>
  );
};
export default HealthReportGenerator;
// const HealthTrendChart = () => {
//   const data = [
//     { month: '9月', score: 82 },
//     { month: '10月', score: 68 },
//     { month: '11月', score: 76 },
//   ];
//
//   return (
//     <ResponsiveContainer width="100%" height={200}>
//       <LineChart data={data}>
//         <XAxis dataKey="month" />
//         <YAxis domain={[0, 100]} />
//         <Line type="monotone" dataKey="score" stroke="#1890ff" strokeWidth={2} />
//       </LineChart>
//     </ResponsiveContainer>
//   );
// };
import React, { useState, useEffect } from 'react';
import {
  Card,
  Table,
  Form,
  Input,
  Select,
  Button,
  DatePicker,
  Modal,
  message,
  Tag,
  Divider,
  Typography,
```

```
  Row,
  Col,
  Statistic,
  Spin,
  Badge,
  Popconfirm,
  Tooltip,
} from 'antd';
import moment from 'moment';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip as ReTooltip,
ResponsiveContainer } from 'recharts';
const { Title, Text } = Typography;
const { Option } = Select;
const { RangePicker } = DatePicker;
const NutritionPlanModule = () => {
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(false);
  const [plans, setPlans] = useState([]);
  const [currentPlan, setCurrentPlan] = useState(null);
  const [visible, setVisible] = useState(false);
  const [reportVisible, setReportVisible] = useState(false);
  const [selectedPlanId, setSelectedPlanId] = useState(null);
  const [chartData, setChartData] = useState([]);
  // 模拟用户信息
  const userInfo = {
    name: '张伟',
    age: 36,
    gender: '男',
    bmi: 24.5,
    condition: ['轻度疲劳', '睡眠质量差'],
  };
  // 营养目标选项
  const goalOptions = [
    { label: '均衡饮食', value: 'balanced' },
    { label: '减脂塑形', value: 'weight_loss' },
    { label: '增肌强化', value: 'muscle_gain' },
    { label: '改善睡眠', value: 'sleep_improvement' },
    { label: '提升精力', value: 'energy_boost' },
  ];
  // 饮食偏好
  const dietPreferenceOptions = [
    { label: '普通饮食', value: 'normal' },
    { label: '低糖饮食', value: 'low_sugar' },
    { label: '素食', value: 'vegetarian' },
    { label: '无麸质', value: 'gluten_free' },
    { label: '高蛋白', value: 'high_protein' },
  ];
  // 初始数据
  useEffect(() => {
    fetchPlans();
```

```
    generateChartMockData();
  }, []);
  const generateChartMockData = () => {
    const data = [];
    for (let i = 0; i < 12; i++) {
      data.push({
        month: `${i + 1}月`,
        score: Math.floor(Math.random() * 20) + 70,
        risk: Math.floor(Math.random() * 15),
      });
    }
    setChartData(data);
  };
  const fetchPlans = () => {
    setLoading(true);
    setTimeout(() => {
      const mockPlans = [
        {
          id: 1,
          userId: 1001,
          startDate: '2025-10-01',
          endDate: '2025-10-31',
          goal: 'sleep_improvement',
          preference: 'low_sugar',
          status: 'completed',
          score: 85,
          riskLevel: 'low',
          createdAt: '2025-09-28T10:00:00Z',
        },
        {
          id: 2,
          userId: 1001,
          startDate: '2025-11-01',
          endDate: '2025-11-30',
          goal: 'balanced',
          preference: 'normal',
          status: 'active',
          score: null,
          riskLevel: 'medium',
          createdAt: '2025-10-28T14:30:00Z',
        },
        {
          id: 3,
          userId: 1001,
          startDate: '2025-12-01',
          endDate: '2025-12-31',
          goal: 'weight_loss',
          preference: 'low_sugar',
          status: 'pending',
          score: null,
```

```
        riskLevel: 'high',
        createdAt: '2025-11-05T09:15:00Z',
      },
    ];
    setPlans(mockPlans);
    setLoading(false);
  }, 800);
};
const handleCreate = () => {
  form.resetFields();
  setCurrentPlan(null);
  setVisible(true);
};
const handleEdit = (record) => {
  setCurrentPlan(record);
  form.setFieldsValue({
    ...record,
    dateRange: [moment(record.startDate), moment(record.endDate)],
  });
  setVisible(true);
};
const handleDelete = (id) => {
  setPlans(plans.filter((plan) => plan.id !== id));
  message.success('营养计划已删除');
};
const handleCancel = () => {
  setVisible(false);
  setReportVisible(false);
};
const onFinish = (values) => {
  const newPlan = {
    ...values,
    id: currentPlan ? currentPlan.id : Date.now(),
    userId: 1001,
    startDate: values.dateRange[0].format('YYYY-MM-DD'),
    endDate: values.dateRange[1].format('YYYY-MM-DD'),
    status: currentPlan ? currentPlan.status : 'pending',
    createdAt: new Date().toISOString(),
  };
  if (currentPlan) {
    setPlans(plans.map((p) => (p.id === newPlan.id ? newPlan : p)));
    message.success('营养计划已更新');
  } else {
    setPlans([newPlan, ...plans]);
    message.success('营养计划已创建');
  }
  setVisible(false);
};
const viewReport = (planId) => {
  setSelectedPlanId(planId);
```

```
    setReportVisible(true);
  };
  const statusMap = {
    active: { text: '进行中', color: 'green' },
    completed: { text: '已完成', color: 'blue' },
    pending: { text: '待开始', color: 'orange' },
  };
  const riskLevelMap = {
    low: { text: '低风险', color: 'success' },
    medium: { text: '中风险', color: 'warning' },
    high: { text: '高风险', color: 'error' },
  };
  const columns = [
    {
      title: '计划 ID',
      dataIndex: 'id',
      key: 'id',
    },
    {
      title: '开始日期',
      dataIndex: 'startDate',
      key: 'startDate',
      render: (text) => moment(text).format('YYYY-MM-DD'),
    },
    {
      title: '结束日期',
      dataIndex: 'endDate',
      key: 'endDate',
      render: (text) => moment(text).format('YYYY-MM-DD'),
    },
    {
      title: '营养目标',
      dataIndex: 'goal',
      key: 'goal',
      render: (value) => goalOptions.find((g) => g.value === value)?.label || '-',
    },
    {
      title: '饮食偏好',
      dataIndex: 'preference',
      key: 'preference',
      render: (value) => dietPreferenceOptions.find((d) => d.value === value)?.label
|| '-',
    },
    {
      title: '状态',
      dataIndex: 'status',
      key: 'status',
      render: (status) => <Tag
color={statusMap[status]?.color}>{statusMap[status]?.text}</Tag>,
    },
```

```
    {
      title: '评分',
      dataIndex: 'score',
      key: 'score',
      render: (score) => (score ? `${score}/100` : '—'),
    },
    {
      title: '风险等级',
      dataIndex: 'riskLevel',
      key: 'riskLevel',
      render: (level) => <Tag
color={riskLevelMap[level]?.color}>{riskLevelMap[level]?.text}</Tag>,
    },
    {
      title: '操作',
      key: 'action',
      render: (_, record) => (
        <span>
          <Button type="link" size="small" onClick={() => handleEdit(record)}>
            编辑
          </Button>
          <Popconfirm title="确定删除此计划吗？" onConfirm={() =>
handleDelete(record.id)}>
            <Button type="link" danger size="small">
              删除
            </Button>
          </Popconfirm>
          <Button type="link" size="small" onClick={() => viewReport(record.id)}>
            查看报告
          </Button>
        </span>
      ),
    },
  ];
  const mockReportData = {
    suggestions: [
      '增加深色蔬菜摄入，如菠菜、西兰花，以补充维生素 K 和叶酸。',
      '减少加工食品摄入，避免反式脂肪酸对心血管的潜在影响。',
      '每日饮水量建议提升至 1.8L 以上，有助于代谢废物排出。',
      '晚餐时间建议提前至 19:00 前，有利于消化系统休息。',
      '可适当补充富含 Omega-3 的食物，如三文鱼、亚麻籽。',
    ],
    riskPrediction: [
      '当前膳食纤维摄入不足，长期可能增加肠道问题风险。',
      '钠摄入偏高，建议控制调味品使用，预防高血压倾向。',
      '蛋白质来源较单一，建议多样化动物与植物蛋白搭配。',
    ],
    nutrientsSummary: {
      protein: 78,
      fat: 65,
```

```
      carbs: 210,
      fiber: 18,
      sodium: 3200,
    },
  };
  // const handleFeedbackSubmit = (feedback) => {
  //   console.log('用户反馈:', feedback);
  //   message.info('感谢您的反馈，我们将持续优化报告内容。');
  // };
  return (
    <div style={{ padding: '24px', background: '#f0f2f5' }}>
      <Card bordered={false}>
        <Title level={2}>个性化营养计划管理</Title>
        <Text type="secondary">为亚健康用户提供科学饮食指导与动态评估</Text>
        <Divider />
        <Row gutter={[16, 16]} style={{ marginBottom: 16 }}>
          <Col span={8}>
            <Card bordered hoverable>
              <Statistic title="总计划数" value={plans.length} />
            </Card>
          </Col>
          <Col span={8}>
            <Card bordered hoverable>
              <Statistic
                title="活跃计划"
                value={plans.filter((p) => p.status === 'active').length}
                prefix={<Badge status="processing" />}
              />
            </Card>
          </Col>
          <Col span={8}>
            <Card bordered hoverable>
              <Statistic
                title="平均评分"
                value={
                  plans.filter((p) => p.score).length > 0
                    ? (plans.filter((p) => p.score).reduce((sum, p) => sum + p.score,
0) / plans.filter((p) => p.score).length).toFixed(1)
                    : 0
                }
                suffix="/100"
              />
            </Card>
          </Col>
        </Row>
        <Card type="inner" title="用户基本信息" style={{ marginBottom: 16 }}>
          <Row>
            <Col span={6}>
              <Text strong>姓名：</Text>
              {userInfo.name}
```

```
            </Col>
            <Col span={6}>
              <Text strong>年龄：</Text>
              {userInfo.age} 岁
            </Col>
            <Col span={6}>
              <Text strong>性别：</Text>
              {userInfo.gender}
            </Col>
            <Col span={6}>
              <Text strong>BMI：</Text>
              {userInfo.bmi}
            </Col>
          </Row>
          <div style={{ marginTop: 8 }}>
            <Text strong>当前状况：</Text>
            {userInfo.condition.map((cond, index) => (
              <Tag key={index} color="magenta">
                {cond}
              </Tag>
            ))}
          </div>
        </Card>
        <Card type="inner" title="历史计划趋势" style={{ marginBottom: 16, height:
300 }}>
          <ResponsiveContainer width="100%" height="100%">
            <LineChart data={chartData}>
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis dataKey="month" />
              <YAxis yAxisId="left" orientation="left" stroke="#1890ff" domain={[70,
100]} label={{ value: '评分', angle: -90, position: 'insideLeft' }} />
              <YAxis    yAxisId="right"    orientation="right"    stroke="#faad14"
domain={[0, 20]} label={{ value: '风险值', angle: 90, position: 'insideRight' }} />
              <ReTooltip />
              <Line yAxisId="left" type="monotone" dataKey="score" stroke="#1890ff"
name="计划评分" />
              <Line yAxisId="right" type="monotone" dataKey="risk" stroke="#faad14"
name="风险指数" />
            </LineChart>
          </ResponsiveContainer>
        </Card>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginBottom: 16 }}>
          <Title level={4} style={{ margin: 0 }}>营养计划列表</Title>
          <Button type="primary" onClick={handleCreate}>
            创建新计划
          </Button>
        </div>
        <Table
          dataSource={plans}
```

```
          columns={columns}
          rowKey="id"
          loading={loading}
          pagination={{ pageSize: 5 }}
        />
        <Modal
          title={currentPlan ? '编辑营养计划' : '创建新营养计划'}
          open={visible}
          onCancel={handleCancel}
          footer={null}
          width={600}
        >
          <Form form={form} layout="vertical" onFinish={onFinish}>
            <Form.Item name="goal" label="营养目标" rules={[{ required: true }]}>
              <Select placeholder="请选择营养目标">
                {goalOptions.map((opt) => (
                  <Option key={opt.value} value={opt.value}>
                    {opt.label}
                  </Option>
                ))}
              </Select>
            </Form.Item>
            <Form.Item name="preference" label="饮食偏好" rules={[{ required:
true }]}>
              <Select mode="multiple" placeholder="请选择饮食偏好">
                {dietPreferenceOptions.map((opt) => (
                  <Option key={opt.value} value={opt.value}>
                    {opt.label}
                  </Option>
                ))}
              </Select>
            </Form.Item>
            <Form.Item name="dateRange" label="计划周期" rules={[{ required:
true }]}>
              <RangePicker format="YYYY-MM-DD" />
            </Form.Item>
            {/*
            <Form.Item name="notes" label="备注">
              <Input.TextArea rows={3} placeholder="可填写特殊饮食要求或过敏源" />
            </Form.Item>
            */}
            <Form.Item>
              <Button type="primary" htmlType="submit" block>
                提交计划
              </Button>
            </Form.Item>
          </Form>
        </Modal>
        <Modal
          title="营养计划报告"
```

```
          open={reportVisible}
          onCancel={handleCancel}
          footer={null}
          width={800}
        >
          {selectedPlanId && (
            <div>
              <Card type="inner" style={{ marginBottom: 16 }}>
                <Title level={4}>计划概览</Title>
                <p><strong> 计 划 周 期 ： </strong>{plans.find(p  =>  p.id  ===
selectedPlanId)?.startDate}     至          {plans.find(p     =>     p.id     ===
selectedPlanId)?.endDate}</p>
                <p><strong>营养目标：</strong>{goalOptions.find(g => g.value ===
plans.find(p => p.id === selectedPlanId)?.goal)?.label}</p>
                <p><strong> 饮 食 偏 好 ： </strong>{plans.find(p  =>  p.id  ===
selectedPlanId)?.preference.map(val =>
                  dietPreferenceOptions.find(d => d.value === val)?.label
                ).join('、')}</p>
                <p><strong> 当 前 评 分 ： </strong>{plans.find(p  =>  p.id  ===
selectedPlanId)?.score || '暂无'} / 100</p>
                <p><strong>风险等级：</strong>
                  <Tag     color={riskLevelMap[plans.find(p     =>     p.id     ===
selectedPlanId)?.riskLevel]?.color}>
                    {riskLevelMap[plans.find(p         =>         p.id         ===
selectedPlanId)?.riskLevel]?.text}
                  </Tag>
                </p>
              </Card>
              <Card type="inner" style={{ marginBottom: 16 }}>
                <Title level={4}>饮食建议</Title>
                <ul>
                  {mockReportData.suggestions.map((s, idx) => (
                    <li key={idx}>{s}</li>
                  ))}
                </ul>
              </Card>
              <Card type="inner" style={{ marginBottom: 16 }}>
                <Title level={4}>风险预测</Title>
                <ul>
                  {mockReportData.riskPrediction.map((r, idx) => (
                    <li key={idx}>
                      <Tag color="error">风险</Tag> {r}
                    </li>
                  ))}
                </ul>
              </Card>
              <Card type="inner">
                <Title level=={4}>营养素摄入统计</Title>
                <Row gutter={[16, 16]}>
                  <Col span={8}>
```

```
                    <Statistic      title="     蛋     白     质          (g)"
value={mockReportData.nutrientsSummary.protein} suffix="/日推荐" />
                  </Col>
                  <Col span={8}>
                    <Statistic        title="      脂      肪              (g)"
value={mockReportData.nutrientsSummary.fat} suffix="/日推荐" />
                  </Col>
                  <Col span={8}>
                    <Statistic    title="   碳   水   化   合   物        (g)"
value={mockReportData.nutrientsSummary.carbs} suffix="/日推荐" />
                  </Col>
                  <Col span={8}>
                    <Statistic     title="    膳    食    纤    维         (g)"
value={mockReportData.nutrientsSummary.fiber} suffix="/日推荐" />
                  </Col>
                  <Col span={8}>
                    <Statistic          title="         钠                  (mg)"
value={mockReportData.nutrientsSummary.sodium} suffix="/日上限" />
                  </Col>
                </Row>
              </Card>
              {/*
              <Divider />
              <Form layout="vertical">
                <Form.Item label="您对本报告的改进建议">
                  <Input.TextArea rows={4} placeholder="欢迎提出宝贵意见..." />
                </Form.Item>
                <Button      type="primary"      block      onClick={()      =>
handleFeedbackSubmit('')}>
                  提交反馈
                </Button>
              </Form>
              */}
            </div>
          )}
        </Modal>
      </Card>
    </div>
  );
};
export default NutritionPlanModule;
import React, { useState, useEffect, useMemo } from 'react';
import {
  Card,
  Row,
  Col,
  Statistic,
  Table,
  Select,
  DatePicker,
```

```
  Button,
  Modal,
  message,
  Spin,
  Tag,
  Tooltip,
  Divider,
  notification,
  Progress,
  Tabs,
  Badge,
} from 'antd';
import {
  LineChart,
  Line,
  BarChart,
  Bar,
  PieChart,
  Pie,
  Cell,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip as ReTooltip,
  Legend,
  ResponsiveContainer,
} from 'recharts';
import moment from 'moment';
import _ from 'lodash';
const { RangePicker } = DatePicker;
const { Option } = Select;
const { TabPane } = Tabs;
// 模拟数据源
const mockUserData = [
  {
    date: '2025-10-01',
    steps: 8400,
    sleepDuration: 7.2,
    heartRate: 76,
    waterIntake: 1800,
    caloriesBurned: 2300,
    mood: 7,
    dietQuality: 6,
  },
  {
    date: '2025-10-02',
    steps: 6100,
    sleepDuration: 6.5,
    heartRate: 79,
    waterIntake: 1500,
```

```
    caloriesBurned: 1950,
    mood: 5,
    dietQuality: 5,
  },
  {
    date: '2025-10-03',
    steps: 10200,
    sleepDuration: 8.0,
    heartRate: 74,
    waterIntake: 2100,
    caloriesBurned: 2700,
    mood: 8,
    dietQuality: 8,
  },
  {
    date: '2025-10-04',
    steps: 4300,
    sleepDuration: 5.8,
    heartRate: 82,
    waterIntake: 1200,
    caloriesBurned: 1600,
    mood: 4,
    dietQuality: 4,
  },
  {
    date: '2025-10-05',
    steps: 9100,
    sleepDuration: 7.5,
    heartRate: 75,
    waterIntake: 1950,
    caloriesBurned: 2400,
    mood: 7,
    dietQuality: 7,
  },
  {
    date: '2025-10-06',
    steps: 7300,
    sleepDuration: 6.9,
    heartRate: 77,
    waterIntake: 1600,
    caloriesBurned: 2100,
    mood: 6,
    dietQuality: 6,
  },
  {
    date: '2025-10-07',
    steps: 11500,
    sleepDuration: 8.2,
    heartRate: 72,
    waterIntake: 2300,
```

```
    caloriesBurned: 2900,
    mood: 9,
    dietQuality: 9,
  },
];
const healthMetricsConfig = {
  steps: { label: '步数', unit: '步', normalRange: [5000, 10000] },
  sleepDuration: { label: '睡眠时长', unit: '小时', normalRange: [7, 9] },
  heartRate: { label: '心率', unit: 'bpm', normalRange: [60, 100] },
  waterIntake: { label: '饮水量', unit: 'ml', normalRange: [1500, 2000] },
  caloriesBurned: { label: '消耗热量', unit: 'kcal', normalRange: [2000, 2500] },
  mood: { label: '心情评分', unit: '/10', normalRange: [6, 10] },
  dietQuality: { label: '饮食质量', unit: '/10', normalRange: [6, 10] },
};
const DataAnalysisDashboard = () => {
  const [loading, setLoading] = useState(true);
  const [dateRange, setDateRange] = useState([
    moment().subtract(6, 'days'),
    moment(),
  ]);
  const [selectedDataSource, setSelectedDataSource] = useState('all');
  const [filteredData, setFilteredData] = useState([]);
  const [abnormalRecords, setAbnormalRecords] = useState([]);
  const [showAlarmModal, setShowAlarmModal] = useState(false);
  useEffect(() => {
    // 模拟数据加载
    setTimeout(() => {
      processUserData();
      checkAbnormalData();
      setLoading(false);
    }, 1000);
  }, [dateRange]);
  const processUserData = () => {
    const start = dateRange[0].format('YYYY-MM-DD');
    const end = dateRange[1].format('YYYY-MM-DD');
    const filtered = mockUserData.filter((item) => {
      const itemDate = item.date;
      return itemDate >= start && itemDate <= end;
    });
    // 数据清洗：移除异常值或补全缺失值（此处简化处理）
    const cleanedData = filtered.map((item) => ({
      ...item,
      mood: item.mood || 5,
      dietQuality: item.dietQuality || 5,
    }));
    setFilteredData(cleanedData);
  };
  const checkAbnormalData = () => {
    const abnormalities = [];
    mockUserData.forEach((record) => {
```

```
      Object.keys(healthMetricsConfig).forEach((key) => {
        const config = healthMetricsConfig[key];
        const value = record[key];
        const [min, max] = config.normalRange;
        if (value < min || value > max) {
          abnormalities.push({
            date: record.date,
            metric: config.label,
            value: value,
            unit: config.unit,
            status: value < min ? '偏低' : '偏高',
          });
        }
      });
    });
    setAbnormalRecords(abnormalities);
    if (abnormalities.length > 0) {
      // notification.warning({
      //   message: '健康数据异常提醒',
      //   description: `发现 ${abnormalities.length} 条异常数据，请及时关注`,
      //   duration: 0,
      // });
    }
  };
  const handleDateChange = (dates) => {
    if (dates) {
      setDateRange(dates);
    }
  };
  const handleDataSourceChange = (value) => {
    setSelectedDataSource(value);
  };
  const showAlarmDetails = () => {
    setShowAlarmModal(true);
  };
  const closeAlarmModal = () => {
    setShowAlarmModal(false);
  };
  const overallStats = useMemo(() => {
    if (filteredData.length === 0) return {};
    const totalSteps = _.sumBy(filteredData, 'steps');
    const avgSleep = _.meanBy(filteredData, 'sleepDuration').toFixed(1);
    const avgHeartRate = _.meanBy(filteredData, 'heartRate').toFixed(0);
    const avgMood = _.meanBy(filteredData, 'mood').toFixed(1);
    const complianceRate =
      filteredData.filter((d) => d.steps >= 5000).length / filteredData.length;
    return {
      totalSteps,
      avgSleep,
      avgHeartRate,
```

```
      avgMood,
      complianceRate,
    };
  }, [filteredData]);
  const chartData = filteredData.map((item) => ({
    name: item.date.slice(-5),
    步数: item.steps / 100,
    睡眠: item.sleepDuration,
    心情: item.mood,
    饮食: item.dietQuality,
  }));
  const pieChartData = [
    { name: '正常数据', value: mockUserData.length - abnormalRecords.length },
    { name: '异常数据', value: abnormalRecords.length },
  ];
  const COLORS = ['#00C49F', '#FF6B6B'];
  const columns = [
    {
      title: '日期',
      dataIndex: 'date',
      key: 'date',
    },
    {
      title: '指标',
      dataIndex: 'metric',
      key: 'metric',
    },
    {
      title: '数值',
      dataIndex: 'value',
      key: 'value',
      render: (text, record) => `${text} ${record.unit}`,
    },
    {
      title: '状态',
      dataIndex: 'status',
      key: 'status',
      render: (text) =>
        text === '偏低' ? (
          <Tag color="orange">{text}</Tag>
        ) : (
          <Tag color="red">{text}</Tag>
        ),
    },
  ];
  // const exportToCSV = () => {
  //   const headers = ['日期', '步数', '睡眠时长', '心率', '饮水量', '消耗热量', '
心情', '饮食'];
  //   const csvContent =
  //     'data:text/csv;charset=utf-8,' +
```

```
  //     headers.join(',') +
  //     '\n' +
  //     filteredData
  //       .map(
  //         (row) =>
  //
`${row.date},${row.steps},${row.sleepDuration},${row.heartRate},${row.waterIntake}
,${row.caloriesBurned},${row.mood},${row.dietQuality}`
  //       )
  //       .join('\n');
  //   const encodedUri = encodeURI(csvContent);
  //   const link = document.createElement('a');
  //   link.setAttribute('href', encodedUri);
  //   link.setAttribute('download', '亚健康数据分析报告.csv');
  //   document.body.appendChild(link);
  //   link.click();
  //   document.body.removeChild(link);
  // };
  const renderHealthTrendCharts = () => (
    <Row gutter={[16, 16]} style={{ marginTop: 20 }}>
      <Col span={24}>
        <Card title="健康趋势分析" bordered={false}>
          <ResponsiveContainer width="100%" height={300}>
            <LineChart data={chartData}>
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis dataKey="name" />
              <YAxis yAxisId="left" />
              <YAxis yAxisId="right" orientation="right" />
              <ReTooltip />
              <Legend />
              <Line
                yAxisId="left"
                type="monotone"
                dataKey="步数"
                stroke="#8884d8"
                name="步数 (x100)"
              />
              <Line
                yAxisId="left"
                type="monotone"
                dataKey="睡眠"
                stroke="#82ca9d"
                name="睡眠时长 (小时)"
              />
              <Line
                yAxisId="right"
                type="monotone"
                dataKey="心情"
                stroke="#ffc658"
                name="心情评分"
```

```
            />
            <Line
              yAxisId="right"
              type="monotone"
              dataKey="饮食"
              stroke="#ff7300"
              name="饮食质量"
            />
          </LineChart>
        </ResponsiveContainer>
      </Card>
    </Col>
    <Col span={12}>
      <Card title="每日步数与心率对比" bordered={false}>
        <ResponsiveContainer width="100%" height={300}>
          <BarChart data={filteredData}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="date" />
            <YAxis yAxisId="left" />
            <YAxis yAxisId="right" orientation="right" />
            <ReTooltip />
            <Legend />
            <Bar yAxisId="left" dataKey="steps" fill="#413ea0" name="步数" />
            <Bar
              yAxisId="right"
              dataKey="heartRate"
              fill="#f00"
              name="心率 (bpm)"
            />
          </BarChart>
        </ResponsiveContainer>
      </Card>
    </Col>
    <Col span={12}>
      <Card title="健康数据分布" bordered={false}>
        <ResponsiveContainer width="100%" height={300}>
          <PieChart>
            <Pie
              data={pieChartData}
              cx="50%"
              cy="50%"
              labelLine={false}
              label={({ name, percent }) =>
                `${name} ${(percent * 100).toFixed(0)}%`
              }
              outerRadius={80}
              fill="#8884d8"
              dataKey="value"
            >
              {pieChartData.map((entry, index) => (
```

```
                <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]}
/>
              ))}
            </Pie>
            <ReTooltip />
          </PieChart>
        </ResponsiveContainer>
      </Card>
    </Col>
  </Row>
);
const renderSummaryCards = () => (
  <Row gutter={[16, 16]} style={{ marginBottom: 20 }}>
    <Col span={6}>
      <Card>
        <Statistic
          title="总步数"
          value={overallStats.totalSteps}
          suffix="步"
          precision={0}
        />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic
          title="平均睡眠"
          value={overallStats.avgSleep}
          suffix="小时"
          precision={1}
        />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic
          title="平均心率"
          value={overallStats.avgHeartRate}
          suffix="bpm"
          precision={0}
        />
      </Card>
    </Col>
    <Col span={6}>
      <Card>
        <Statistic
          title="平均心情"
          value={overallStats.avgMood}
          suffix="/10"
          precision={1}
```

```
        />
      </Card>
    </Col>
  </Row>
);
const renderComplianceProgress = () => {
  const rate = overallStats.complianceRate || 0;
  let status = 'success';
  if (rate < 0.5) status = 'exception';
  else if (rate < 0.8) status = 'active';
  return (
    <Card title="健康达标率" style={{ marginTop: 20 }}>
      <Progress
        percent={Math.round(rate * 100)}
        status={status}
        strokeColor={{
          '0%': '#108ee9',
          '100%': '#87d068',
        }}
      />
      <p style={{ marginTop: 10, color: '#999' }}>
        近期步数达标天数占比：{Math.round(rate * 100)}%
      </p>
    </Card>
  );
};
if (loading) {
  return (
    <div style={{ textAlign: 'center', marginTop: 50 }}>
      <Spin size="large" tip="数据加载中..." />
    </div>
  );
}
return (
  <div style={{ padding: 24, background: '#f5f5f5', minHeight: '100vh' }}>
    <Card
      title="亚健康数据统计分析系统"
      extra={
        <Button type="primary" onClick={showAlarmDetails}>
          异常提醒{' '}
          <Badge
            count={abnormalRecords.length}
            overflowCount={9}
            style={{ backgroundColor: '#f50' }}
          />
        </Button>
      }
    >
      <Row gutter={[16, 16]} align="middle">
        <Col span={8}>
```

```
          <RangePicker
            value={dateRange}
            onChange={handleDateChange}
            allowClear={false}
          />
        </Col>
        <Col span={8}>
          <Select
            value={selectedDataSource}
            style={{ width: '100%' }}
            onChange={handleDataSourceChange}
          >
            <Option value="all">全部数据源</Option>
            <Option value="wearable">可穿戴设备</Option>
            <Option value="manual">手动输入</Option>
            <Option value="app">健康 APP 同步</Option>
          </Select>
        </Col>
        <Col span={8} style={{ textAlign: 'right' }}>
          {/* <Button onClick={exportToCSV}>导出报告</Button> */}
        </Col>
      </Row>
      {renderSummaryCards()}
      <Tabs defaultActiveKey="trends">
        <TabPane tab="趋势分析" key="trends">
          {renderHealthTrendCharts()}
          {renderComplianceProgress()}
        </TabPane>
        <TabPane tab="原始数据" key="raw">
          <Table
            dataSource={filteredData}
            rowKey="date"
            pagination={{ pageSize: 5 }}
          />
        </TabPane>
      </Tabs>
    </Card>
    <Modal
      title="健康数据异常提醒"
      open={showAlarmModal}
      onCancel={closeAlarmModal}
      footer={[
        <Button key="ok" type="primary" onClick={closeAlarmModal}>
          知道了
        </Button>,
      ]}
      width={800}
    >
      <p>
        检测到您的部分健康数据存在异常，建议及时调整生活习惯或咨询专业医生。
```

```
        </p>
        <Divider />
        <Table
          dataSource={abnormalRecords}
          columns={columns}
          rowKey={(record) => `${record.date}-${record.metric}`}
          pagination={{ pageSize: 5 }}
          scroll={{ y: 300 }}
        />
      </Modal>
    </div>
  );
};
export default DataAnalysisDashboard;
// class LegacyDataView extends React.Component {
//   constructor(props) {
//     super(props);
//     this.state = {
//       data: [],
//       loading: true,
//     };
//   }
//   componentDidMount() {
//     setTimeout(() => {
//       this.setState({ data: mockUserData, loading: false });
//     }, 1500);
//   }
//   render() {
//     const { loading, data } = this.state;
//     if (loading) return <Spin />;
//     return (
//       <Table
//         dataSource={data}
//         columns={[
//           { title: '日期', dataIndex: 'date' },
//           { title: '步数', dataIndex: 'steps' },
//           { title: '睡眠', dataIndex: 'sleepDuration' },
//         ]}
//       />
//     );
//   }
// }
// const SimpleChart = () => {
//   const data = [
//     { name: 'A', value: 400 },
//     { name: 'B', value: 300 },
//     { name: 'C', value: 600 },
//   ];
//   return (
//     <ResponsiveContainer width="100%" height={200}>
```

```
//       <BarChart data={data}>
//         <Bar dataKey="value" fill="#413ea0" />
//       </BarChart>
//     </ResponsiveContainer>
//   );
// };
import React, { useState, useEffect, useCallback } from 'react';
import {
  Layout,
  Menu,
  Card,
  Button,
  Modal,
  Form,
  Input,
  Select,
  Radio,
  message,
  Typography,
  Divider,
  Avatar,
  List,
  Comment,
  Row,
  Col,
  Tag,
  Rate,
  Space,
  Spin,
  InputNumber,
} from 'antd';
import {
  HomeOutlined,
  MessageOutlined,
  ReadOutlined,
  UserOutlined,
  StarOutlined,
  ClockCircleOutlined,
  CalendarOutlined,
  SendOutlined,
  LoadingOutlined,
  SmileOutlined,
  CrownOutlined,
} from '@ant-design/icons';
const { Header, Content, Sider } = Layout;
const { Title, Paragraph, Text } = Typography;
const { TextArea } = Input;
const { Option } = Select;
// 健康咨询模块主入口组件
const HealthConsultationModule = () => {
```

```
  const [currentView, setCurrentView] = useState('chat');
  const [user, setUser] = useState({ name: '张三', avatar: '', level: '普通用户' });
  const [loading, setLoading] = useState(false);
  const [isModalVisible, setIsModalVisible] = useState(false);
  const menuItems = [
    { key: 'chat', label: '在线咨询', icon: <MessageOutlined /> },
    { key: 'articles', label: '健康科普', icon: <ReadOutlined /> },
    { key: 'profile', label: '我的咨询', icon: <UserOutlined /> },
  ];
  const handleMenuClick = (e) => {
    setCurrentView(e.key);
  };
  const showFeedbackModal = () => {
    setIsModalVisible(true);
  };
  const handleCancel = () => {
    setIsModalVisible(false);
  };
  const handleFinishFeedback = (values) => {
    console.log('提交反馈:', values);
    message.success('感谢您的反馈！');
    setIsModalVisible(false);
  };
  return (
    <Layout style={{ minHeight: '100vh', backgroundColor: '#f5f7fa' }}>
      <Header style={{ background: '#fff', padding: 0, boxShadow: '0 2px 8px
rgba(0,0,0,0.1)' }}>
        <div style={{ display: 'flex', alignItems: 'center', paddingLeft: 20 }}>
          <CrownOutlined style={{ fontSize: 24, color: '#1890ff', marginRight: 12 }}
/>
          <Title level={3} style={{ margin: 0, color: '#1890ff' }}>
            亚健康数据统计分析系统
          </Title>
        </div>
      </Header>
      <Layout>
        <Sider width={220} style={{ background: '#fff', borderRight: '1px solid
#e8e8e8' }}>
          <div style={{ padding: 20 }}>
            <Avatar size={64} icon={<UserOutlined />} style={{ marginBottom: 12 }}
/>
            <Title level={5} style={{ margin: 0 }}>{user.name}</Title>
            <Tag color="blue">{user.level}</Tag>
          </div>
          <Menu
            mode="inline"
            selectedKeys={[currentView]}
            onClick={handleMenuClick}
            items={menuItems}
          />
```

```
          <Divider style={{ margin: '12px 0' }} />
          <Button type="dashed" block onClick={showFeedbackModal}>
            提交建议
          </Button>
        </Sider>
        <Layout style={{ padding: '24px' }}>
          <Content style={{ background: '#fff', padding: 24, borderRadius: 8,
boxShadow: '0 2px 8px rgba(0,0,0,0.1)' }}>
            {currentView === 'chat' && <ConsultationChat />}
            {currentView === 'articles' && <HealthArticles />}
            {currentView === 'profile' && <UserProfileConsultations />}
          </Content>
        </Layout>
      </Layout>
      {/* 反馈模态框 */}
      <Modal
        title="您对我们的服务有建议吗？"
        open={isModalVisible}
        onCancel={handleCancel}
        footer={null}
        destroyOnClose
      >
        <Form onFinish={handleFinishFeedback} layout="vertical">
          <Form.Item
            name="satisfaction"
            label="满意度评分"
            rules={[{ required: true, message: '请打分' }]}
          >
            <Rate allowHalf />
          </Form.Item>
          <Form.Item
            name="content"
            label="您的建议"
            rules={[{ required: true, message: '请输入建议内容' }]}
          >
            <TextArea rows={4} placeholder="请输入您的宝贵意见..." />
          </Form.Item>
          <Form.Item>
            <Button type="primary" htmlType="submit" block>提交</Button>
          </Form.Item>
        </Form>
      </Modal>
    </Layout>
  );
};
// 在线咨询聊天界面
const ConsultationChat = () => {
  const [messages, setMessages] = useState([]);
  const [inputValue, setInputValue] = useState('');
  const [sending, setSending] = useState(false);
```

```
  const [expertInfo, setExpertInfo] = useState({
    name: '李医生',
    title: '资深健康管理师',
    online: true,
    responseTime: '1 分钟内回复',
  });
  // 模拟初始消息
  useEffect(() => {
    setMessages([
      {
        id: 1,
        type: 'system',
        content: '您好，我是您的健康顾问李医生，我可以为您提供运动、饮食、睡眠等方面的建议。',
        timestamp: new Date(Date.now() - 120000),
      },
      {
        id: 2,
        type: 'system',
        content: '请问您今天想了解哪方面的健康问题呢？',
        timestamp: new Date(Date.now() - 110000),
      },
    ]);
  }, []);
  const handleSendMessage = () => {
    if (!inputValue.trim()) return;
    const userMsg = {
      id: messages.length + 1,
      type: 'user',
      content: inputValue,
      timestamp: new Date(),
    };
    setMessages((prev) => [...prev, userMsg]);
    setInputValue('');
    setSending(true);
    // 模拟 AI 回复延迟
    setTimeout(() => {
      const replies = [
        '感谢您的提问。关于这个问题，建议您每天保持 30 分钟的中等强度运动，比如快走或慢跑。',
        '在饮食方面，建议减少高油高盐食物摄入，多吃蔬菜水果和优质蛋白。',
        '睡眠质量对亚健康状态影响很大，建议您每天固定时间入睡，避免睡前使用电子设备。',
        '您可以尝试进行深呼吸练习或冥想，有助于缓解压力和改善情绪。',
      ];
      const replyMsg = {
        id: messages.length + 2,
        type: 'system',
        content: replies[Math.floor(Math.random() * replies.length)],
        timestamp: new Date(),
```

```
      };
      setMessages((prev) => [...prev, replyMsg]);
      setSending(false);
    }, 1500);
  };
  const handleKeyPress = (e) => {
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault();
      handleSendMessage();
    }
  };
  return (
    <div style={{ height: 'calc(100vh - 200px)', display: 'flex', flexDirection:
'column' }}>
      <Card bordered={false} style={{ marginBottom: 16 }}>
        <Space align="center">
          <Avatar size="large" style={{ backgroundColor: '#1890ff' }}>
            {expertInfo.name.charAt(0)}
          </Avatar>
          <div>
            <Title level={5} style={{ margin: 0 }}>{expertInfo.name}</Title>
            <Text type="secondary">{expertInfo.title}</Text>
            <br />
            <Tag color={expertInfo.online ? 'green' : 'default'} style={{ marginTop:
4 }}>
              {expertInfo.online ? '在线' : '离线'}
            </Tag>
            <Text type="secondary" style={{ marginLeft: 8 }}>
              {expertInfo.responseTime}
            </Text>
          </div>
        </Space>
      </Card>
      <div
        style={{
          flex: 1,
          overflowY: 'auto',
          padding: '16px 0',
          border: '1px solid #e8e8e8',
          borderRadius: 8,
          marginBottom: 16,
        }}
      >
        {messages.map((msg) => (
          <div
            key={msg.id}
            style={{
              display: 'flex',
              justifyContent: msg.type === 'user' ? 'flex-end' : 'flex-start',
              marginBottom: 12,
```

```
              paddingRight: msg.type === 'user' ? 10 : 0,
              paddingLeft: msg.type === 'user' ? 0 : 10,
            }}
          >
            {msg.type === 'system' && (
              <Avatar size="small" style={{ backgroundColor: '#1890ff', marginRight:
8 }}>
                {expertInfo.name.charAt(0)}
              </Avatar>
            )}
            <div
              style={{
                maxWidth: '70%',
                padding: '8px 12px',
                borderRadius: 18,
                backgroundColor: msg.type === 'user' ? '#1890ff' : '#f0f2f5',
                color: msg.type === 'user' ? '#fff' : 'rgba(0,0,0,0.85)',
                wordBreak: 'break-word',
              }}
            >
              {msg.content}
              <div
                style={{
                  fontSize: 12,
                  textAlign: msg.type === 'user' ? 'right' : 'left',
                  marginTop: 4,
                  opacity: 0.6,
                }}
              >
                {msg.timestamp.toLocaleTimeString([], { hour: '2-digit', minute:
'2-digit' })}
              </div>
            </div>
            {msg.type === 'user' && <Avatar size="small" icon={<UserOutlined />}
style={{ marginLeft: 8 }} />}
          </div>
        ))}
        {sending && (
          <div style={{ display: 'flex', justifyContent: 'flex-start', marginBottom:
12, paddingLeft: 10 }}>
            <Avatar size="small" style={{ backgroundColor: '#1890ff', marginRight:
8 }}>
              {expertInfo.name.charAt(0)}
            </Avatar>
            <div style={{ padding: '8px 12px', borderRadius: 18, backgroundColor:
'#f0f2f5' }}>
              <LoadingOutlined spin /> 正在输入...
            </div>
          </div>
        )}
```

```
      </div>
      <div style={{ display: 'flex', alignItems: 'center' }}>
        <Input
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          onKeyPress={handleKeyPress}
          placeholder="请输入您的问题..."
          style={{ flex: 1, marginRight: 8 }}
          disabled={sending}
        />
        <Button
          type="primary"
          icon={<SendOutlined />}
          onClick={handleSendMessage}
          loading={sending}
          disabled={!inputValue.trim()}
        >
          发送
        </Button>
      </div>
    </div>
  );
};
// 健康科普文章列表
const HealthArticles = () => {
  const [articles, setArticles] = useState([
    {
      id: 1,
      title: '如何科学改善睡眠质量',
      summary: '睡眠是恢复体力和维持健康的重要环节。本文介绍 5 种有效提升睡眠质量的方法...',
      author: '王医生',
      date: '2025-10-15',
      category: '睡眠调整',
      readCount: 2340,
      cover: 'https://via.placeholder.com/300x180?text=Sleep+Tips',
      liked: false,
    },
    {
      id: 2,
      title: '亚健康人群的饮食指南',
      summary: '针对长期疲劳、免疫力下降等问题，合理膳食是关键。推荐 10 种适合亚健康人群的食物...',
      author: '营养师刘',
      date: '2025-10-20',
      category: '饮食建议',
      readCount: 3120,
      cover: 'https://via.placeholder.com/300x180?text=Diet+Guide',
      liked: true,
    },
```

```
    {
      id: 3,
      title: '办公室族的简易运动方案',
      summary: '久坐办公容易导致肩颈酸痛和代谢减缓。每天 10 分钟，轻松缓解身体不适...',
      author: '健身教练赵',
      date: '2025-10-25',
      category: '运动指导',
      readCount: 1890,
      cover: 'https://via.placeholder.com/300x180?text=Office+Exercise',
      liked: false,
    },
  ]);
  const categories = ['全部', '睡眠调整', '饮食建议', '运动指导'];
  const [activeCategory, setActiveCategory] = useState('全部');
  const [filteredArticles, setFilteredArticles] = useState(articles);
  useEffect(() => {
    if (activeCategory === '全部') {
      setFilteredArticles(articles);
    } else {
      setFilteredArticles(articles.filter((a) => a.category === activeCategory));
    }
  }, [activeCategory, articles]);
  const handleLike = useCallback((id) => {
    setArticles((prev) =>
      prev.map((a) => (a.id === id ? { ...a, liked: !a.liked, readCount: a.readCount + (a.liked ? -1 : 1) } : a))
    );
  }, []);
  return (
    <div>
      <Title level={3}>健康科普知识</Title>
      <Paragraph type="secondary">专业医师团队为您带来实用的健康生活指导</Paragraph>
      <Row gutter={[16, 16]} style={{ marginBottom: 24 }}>
        {categories.map((cat) => (
          <Col key={cat}>
            <Tag
              color={activeCategory === cat ? 'blue' : 'default'}
              style={{ cursor: 'pointer', padding: '6px 12px', fontSize: 14 }}
              onClick={() => setActiveCategory(cat)}
            >
              {cat}
            </Tag>
          </Col>
        ))}
      </Row>
      <List
        grid={{ gutter: 16, xs: 1, sm: 2, md: 3 }}
        dataSource={filteredArticles}
```

```
        renderItem={(item) => (
          <List.Item>
            <Card
              hoverable
              cover={<img alt={item.title} src={item.cover} style={{ height: 180,
objectFit: 'cover' }} />}
              actions={[
                <Space key="read">
                  <ClockCircleOutlined /> {item.date}
                </Space>,
                <Space key="views">
                  <StarOutlined /> {item.readCount}
                </Space>,
                <Space key="like" onClick={() => handleLike(item.id)}
style={{ cursor: 'pointer' }}>
                  <HeartOutlined style={{ color: item.liked ? '#eb2f96' :
'rgba(0,0,0,0.25)' }} />
                  {item.liked ? '已收藏' : '收藏'}
                </Space>,
              ]}
            >
              <Card.Meta
                title={<a href="#article">{item.title}</a>}
                description={
                  <>
                    <Text type="secondary" ellipsis={{ tooltip: item.summary }}>
                      {item.summary}
                    </Text>
                    <div style={{ marginTop: 8 }}>
                      <Tag color="processing">{item.category}</Tag>
                      <Text type="secondary" style={{ marginLeft: 8 }}>
                        {item.author}
                      </Text>
                    </div>
                  </>
                }
              />
            </Card>
          </List.Item>
        )}
      />
    </div>
  );
};
// 用户个人咨询记录
const UserProfileConsultations = () => {
  const [consultations, setConsultations] = useState([
    {
      id: 1,
      question: '最近总是失眠，怎么办？',
```

```
      answer: '建议您建立规律作息，晚上避免使用手机，可以尝试冥想放松。必要时可咨
询专业医生。',
      date: '2025-10-10',
      status: '已回复',
      rating: 5,
    },
    {
      id: 2,
      question: '工作压力大，情绪低落怎么调节？',
      answer: '推荐每天进行 15 分钟正念呼吸练习，同时保证充足睡眠和适度运动，有助于
缓解压力。',
      date: '2025-10-05',
      status: '已回复',
      rating: 4,
    },
    {
      id: 3,
      question: '想开始锻炼，但不知道从何入手',
      answer: '可以从每天快走 30 分钟开始，逐步增加强度。注意热身和拉伸，避免受伤。
',
      date: '2025-09-28',
      status: '已回复',
      rating: 5,
    },
  ]);
  const [newQuestion, setNewQuestion] = useState('');
  const [submitting, setSubmitting] = useState(false);
  const handleSubmitQuestion = () => {
    if (!newQuestion.trim()) {
      message.warning('请输入您的问题');
      return;
    }
    setSubmitting(true);
    setTimeout(() => {
      const newConsult = {
        id: consultations.length + 1,
        question: newQuestion,
        answer: '',
        date: new Date().toISOString().split('T')[0],
        status: '等待回复',
        rating: null,
      };
      setConsultations((prev) => [newConsult, ...prev]);
      setNewQuestion('');
      setSubmitting(false);
      message.success('问题已提交，专家将在 24 小时内回复');
    }, 800);
  };
  return (
    <div>
```

```
      <Title level={3}>我的健康咨询记录</Title>
      <Paragraph type="secondary">查看历史咨询与专家回复</Paragraph>
      <Card title="提出新问题" style={{ marginBottom: 24 }}>
        <TextArea
          value={newQuestion}
          onChange={(e) => setNewQuestion(e.target.value)}
          placeholder="例如：我最近总是感到疲劳，有什么改善建议吗？"
          rows={4}
          style={{ marginBottom: 16 }}
        />
        <Button type="primary" onClick={handleSubmitQuestion} loading={submitting}
disabled={!newQuestion.trim()}>
          提交问题
        </Button>
      </Card>
      <List
        dataSource={consultations}
        renderItem={(item) => (
          <List.Item>
            <List.Item.Meta
              title={
                <Space>
                  <Text strong>{item.question}</Text>
                  <Tag color={item.status === '已回复' ? 'success' :
'warning'}>{item.status}</Tag>
                </Space>
              }
              description={
                <>
                  <Text type="secondary">提问时间：{item.date}</Text>
                  {item.answer && (
                    <>
                      <Divider style={{ margin: '8px 0' }} />
                      <Comment
                        author={<Text strong>健康顾问</Text>}
                        content={<Text>{item.answer}</Text>}
                        datetime={<span>{item.date}</span>}
                      />
                      {item.rating && (
                        <div style={{ marginTop: 8 }}>
                          <Text>评分：</Text>
                          <Rate value={item.rating} disabled />
                        </div>
                      )}
                    </>
                  )}
                </>
              }
            />
          </List.Item>
```

```
        )}
      />
    </div>
  );
};
// function HeartOutlined(props) {
//   return <SmileOutlined {...props} />;
// }
const HeartOutlined = (props) => <SmileOutlined {...props} />;
export default HealthConsultationModule;
```